Practical:06	Implementation of chain matrix multiplication using dynamic programming.
__/__ /2026	

Objective:

- To implement the Matrix Chain Multiplication problem using Dynamic Programming.
- To find the minimum number of scalar multiplications required to multiply a chain of matrices.
- To understand how Dynamic Programming avoids repeated calculations.

Program:

```
#include <iostream>
#include <climits>
using namespace std;

// Function to find minimum multiplication cost
void matrixChainMultiplication(int p[], int n)
{
   int m[n][n];

   // Cost is 0 when multiplying one matrix
   for (int i = 1; i < n; i++)
      m[i][i] = 0;

   // length is the chain length
   for (int length = 2; length < n; length++)
   {
      for (int i = 1; i < n - length + 1; i++)
      {
         int j = i + length - 1;
         m[i][j] = INT_MAX;

         for (int k = i; k < j; k++)
         {
```

```
            int cost = m[i][k] +
                    m[k + 1][j] +
                    p[i - 1] * p[k] * p[j];

            if (cost < m[i][j])
                m[i][j] = cost;
        }
    }
}

    cout << "Minimum number of scalar multiplications: "
        << m[1][n - 1] << endl;
}

int main()
{
    int n;

    cout << "Enter number of matrices: ";
    cin >> n;

    int p[n + 1];

    cout << "Enter dimensions of matrices:\n";

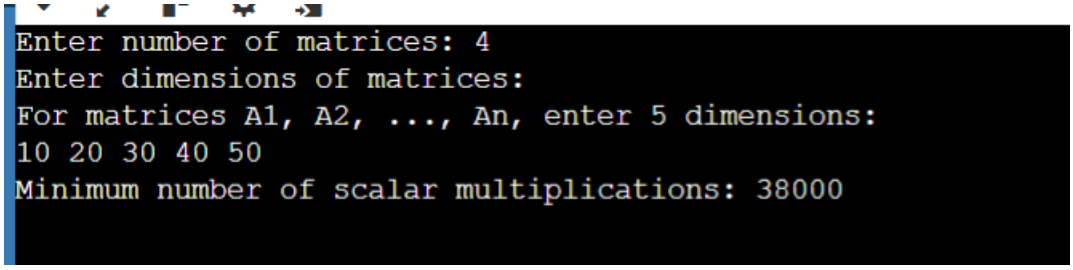
    cout << "For matrices A1, A2, ..., An, enter "
        << n + 1 << " dimensions:\n";

    for (int i = 0; i <= n; i++)
        cin >> p[i];

    matrixChainMultiplication(p, n + 1);
```

```
    return 0;
}
```

Output:



Analysis:

- Dynamic Programming avoids recalculating the same subproblems.
- All possible parenthesizations are considered through the split point k.
- The algorithm finds the optimal multiplication order, not the actual multiplication of matrices.
- It is more efficient than checking every possible parenthesization separately.

For a chain of n matrices, the algorithm uses a DP table $C(i,j)$, where $C(i,j)$stores the minimum multiplication cost for matrices $A_i$through A_j.

Time Complexity

$$\boxed{O(n^3)}$$

There are approximately $O(n^2)$entries in the DP table, and for each entry, all possible split positions are checked, giving $O(n)$work.

Space Complexity

$$\boxed{O(n^2)}$$

The $n \times n$DP table is required to store the minimum costs.

Conclusion:

The Matrix Chain Multiplication problem was successfully implemented using Dynamic Programming. The program determines the optimal order of matrix multiplication that requires the minimum number of scalar multiplications. This method reduces unnecessary calculations by storing previously calculated results.